

WELLS FARGO · CYBER AI PLATFORM

Cyber AI Platform

Consolidated Design Specification

Vulnerability Chain Analysis MVP · Governed Workflow Architecture
Graph RAG · NL2SQL / NL2Cypher / NL2GQL · Domain Agent Framework

VERSION	1.0 (design baseline)
DATE	July 2026
EXECUTION MODEL	Governed workflow mode — no unrestricted dynamic agent execution
STACK	Python / FastAPI · Temporal · Postgres · Redis · Kafka · Graph DB · OpenSearch
MVP SCOPE	vulnerability_chaining_workflow with approval-gated Jira epic/story/task creation

- Part A** — MVP logical layout, high-level architecture, and interface specification (13 services)
- Part B** — Graph RAG subsystem, governed NL2Query pattern, and domain agent onboarding & composition
- Part C** — Agent vs. workflow relationship model

Contents

Part A — MVP Architecture & Interface Specification

1. Logical Architecture
2. High-Level Architecture
3. Workflow Definition — vulnerability_chaining_workflow
4. Interface Specification
5. Event Contracts (Kafka)
6. Cross-Cutting Contracts
7. Data Store Ownership
8. Build Phases → Interface Readiness
9. Design Principles (restated as testable assertions)

Part B — Graph RAG, NL2Query, and the Domain Agent Framework

0. Positioning — nothing bypasses the substrate
1. Graph RAG Subsystem
2. NL2SQL / NL2Cypher / NL2GQL — the governed query pattern
3. Domain Agent Framework
4. Invariants — extended set

Part C — Agent vs. Workflow: Relationship Model

- C.1 The relationship, stated precisely
- C.2 Why two abstractions when the MVP binds them 1:1
- C.3 Registry mapping

Part A — MVP Architecture & Interface Specification

1. Logical Architecture

The platform is organized into six logical planes. Every request traverses the planes top-down; no plane may bypass the plane below it. This is the enforcement backbone: **the only path to enterprise data and enterprise actions runs through the governance plane and the tool gateway.**

L1	EXPERIENCE PLANE chat-api-service Request intake, SSE/WebSocket streaming, workflow status
L2	CONTEXT PLANE context-api Identity, tenant, session, conversation history, memory facts
L3	CONTROL PLANE (governance – "what exists" vs "what is allowed") orchestrator-service → intent → approved workflow registry-api → what exists and is approved (versioned) policy-api → is this allowed now, with what limits
L4	EXECUTION PLANE workflow-runtime-service → Temporal (durable state, signals) agent-runtime-service → vulnerability_chaining_agent + skills tool-gateway-service → sole egress to tools & data
L5	DOMAIN & MODEL PLANE cyber-domain-api → data products, graph paths, evidence model-gateway-service → LLM routing, embeddings, token budgets
L6	ASSURANCE PLANE (cross-cutting) verification-response-service → grounding, quality gate, redact audit-observability-service → audit, traces, cost, tool logs notification-service → approvals, Jira draft review

1.1 Plane invariants

#	INVARIANT	ENFORCED BY
I1	No service invokes a tool or enterprise system except through <code>tool-gateway-service</code> .	Network policy + tool-gateway authN
I2	No workflow starts without a <code>policy-api</code> allow decision bound to a <code>registry-api</code> definition version.	workflow-runtime-service
I3	LLM output never mutates state directly. All writes (Jira creation) are deterministic activities gated by human approval signals.	Temporal + policy-api
I4	<code>verification-response-service</code> has no tool or database egress. It proposes; <code>workflow-runtime-service</code> executes through the governed path.	Network policy
I5	Every cross-service call carries the propagated execution context (§6.1) and emits an audit event.	Middleware + audit-observability-service
I6	Registry definitions are immutable per version. Policy references definitions by <code>(id, version)</code> , never by mutable pointers.	registry-api

1.2 Control-plane split: policy-api vs registry-api

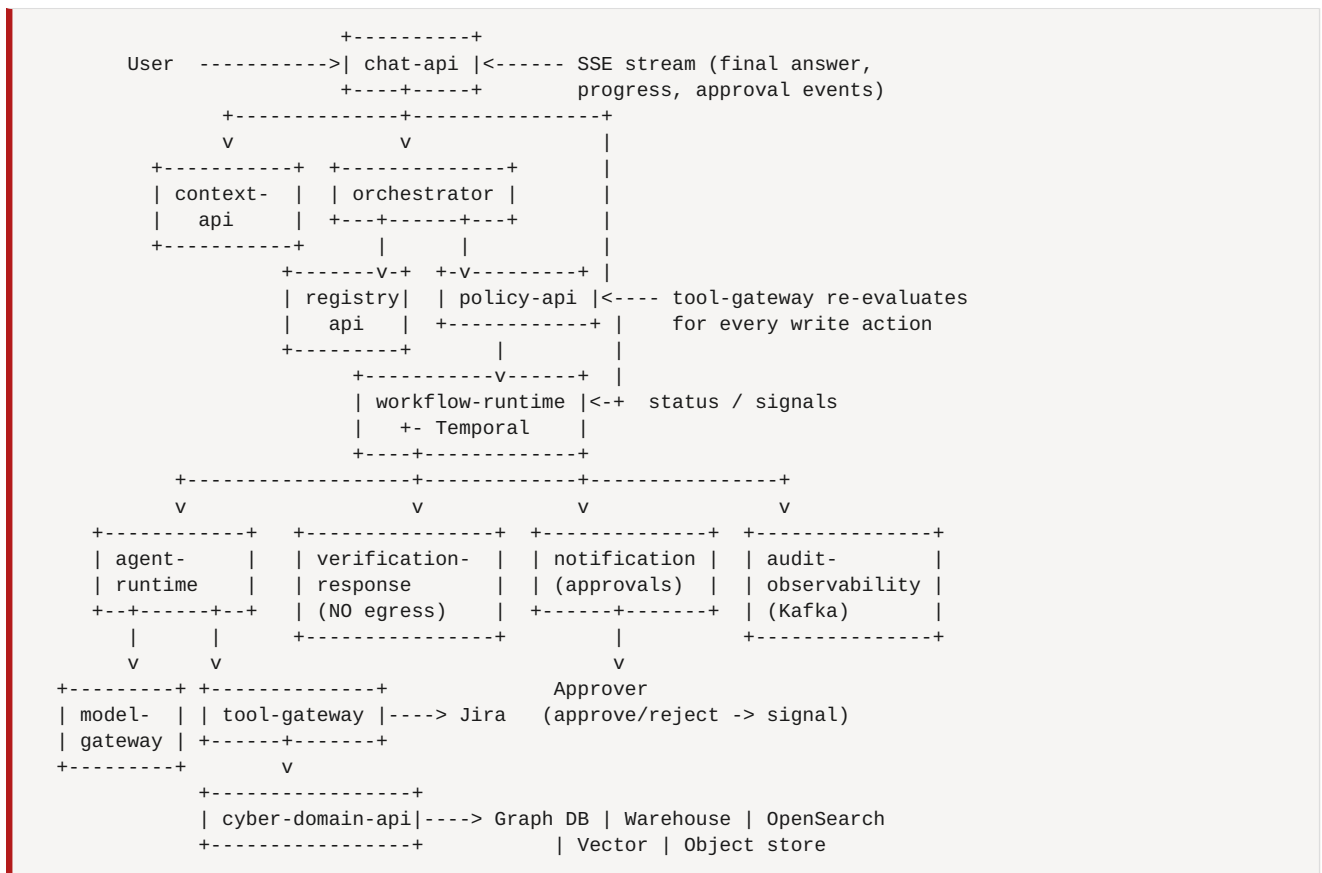
	REGISTRY-API	POLICY-API
Question	<i>What exists and is approved?</i>	<i>Is this allowed, right now, for this principal, and under what constraints?</i>

	REGISTRY-API	POLICY-API
Data	Workflow / agent / skill / tool / model / prompt / data-product definitions, versioned	Authorization rules, tenant boundaries, masking rules, approval policy, obligations
Mutability	Append-only versions; approval lifecycle	Policy versions with effective dates
Consumers	orchestrator, workflow-runtime, agent-runtime, tool-gateway	Every service that performs or requests an action
Failure mode	Fail closed (unknown capability = not executable)	Fail closed (no decision = deny)

Example composition: registry says `graph_query_tool@2` is an approved capability with schema X. Policy says *this* user in *this* tenant may invoke it only with `max_depth=4, read_only=true, pii_mode=masked`. The tool gateway enforces the intersection.

2. High-Level Architecture

2.1 Component diagram



2.2 Sync vs async boundaries

INTERACTION	MODE	RATIONALE
chat-api → orchestrator → registry/policy	Sync REST	Sub-second routing decision; fail fast
workflow-runtime → Temporal	Temporal SDK (gRPC)	Durable execution owner
Temporal activities → agent-runtime / tool-gateway	Sync REST with Temporal retry policy	Retries, timeouts, heartbeats owned by Temporal, not ad-hoc client retry

INTERACTION	MODE	RATIONALE
Approval	Async — Temporal signal via workflow-runtime	Human-in-the-loop is unbounded latency; workflow parks durably
Progress to user	Redis stream → SSE fanout from chat-api	Decouples workflow tempo from client connections
Audit, tool-call logs, cost events	Kafka fire-and-forget with local buffer	Audit must never block execution path; buffered to survive broker outage

2.3 Trust zones

```

Zone A (user-facing):    chat-api
Zone B (control):       context-api, orchestrator, registry-api, policy-api
Zone C (execution):     workflow-runtime, agent-runtime, Temporal workers
Zone D (guarded egress): tool-gateway ← only zone with routes to Zone E
Zone E (enterprise):    cyber-domain-api, Jira, Graph DB, Warehouse
Zone F (assurance):     verification (no egress), audit, notification

```

Service-to-service authentication: mTLS + workload identity; user identity propagated as a signed context token (§6.1), never as a raw bearer replay.

2.4 Runtime flow — happy path

```

1. POST /v1/chat/messages/stream          chat-api
2. POST /v1/context/resolve              context-api → execution context
3. POST /v1/orchestrate                  orchestrator → intent = vulnerability_chain
4. GET /v1/workflows/vulnerability_chaining_workflow registry-api → definition v1
5. POST /v1/policy/evaluate              policy-api → ALLOW + constraints
6. POST /v1/workflows/run                workflow-runtime → Temporal start
7. Temporal executes activities 1-10 (§3) agent-runtime / tool-gateway / cyber-domain
8. POST /v1/verify                      verification → quality gate PASS
9. Final answer streamed via Redis → SSE chat-api

```

2.5 Runtime flow — approval-gated Jira creation

```

1. Workflow reaches jira_epic_draft / story / task draft activities (auto-allowed)
2. Drafts persisted (Postgres metadata + object store payload)
3. POST /v1/approvals                    notification-service → approval_id
4. Workflow parks on Temporal signal `jira_approval` (durable, no timeout leak)
5. Approver reviews drafts, approves/rejects via notification channel
6. POST /v1/executions/{id}/signal       workflow-runtime → Temporal signal
7. create_jira_activity → tool-gateway POST /v1/jira/issues
8. tool-gateway re-evaluates policy (require_for_create: evidence_refs, risk_score,
   affected_assets, remediation_reason, owner_verification) → creates in Jira
9. Audit event with approval_id, decision_id, jira issue keys

```

Approval is enforced **twice**: once at the workflow level (signal required to proceed) and once at the gateway (policy re-evaluation at creation time). A stale or forged signal cannot create tickets without a fresh policy allow.

2.6 Runtime flow — verification quality loop

```

draft → verification-response-service
  checks: evidence refs resolve · chain path validity · data-impact grounding ·
          risk explanation completeness · Jira draft contract (§7.4) · redaction
PASS → final response (or approval request if Jira creation requested)
FAIL → verification returns improvement_actions[]
      workflow-runtime executes them through governed path (max_iterations = 2)
      verification NEVER calls tools or databases itself (invariant I4)

```

3. Workflow Definition — `vulnerability_chaining_workflow`

Temporal workflow; each step is an activity with an explicit retry policy and timeout. All tool access inside activities goes through tool-gateway.

#	ACTIVITY	SKILL	OUTPUT ARTIFACT
1	<code>resolve_entity_activity</code>	<code>entity_resolution_skill</code>	<code>EntityContext</code>
2	<code>retrieve_vulnerability_findings_activity</code>	<code>vulnerability_finding_retrieval_skill</code>	<code>Finding[]</code> (high/critical, tenant-scoped)
3	<code>enrich_cve_activity</code>	<code>cve_exploitability_enrichment_skill</code>	<code>Finding[]</code> + CVSS/EPSS/KEV/exploit maturity
4	<code>map_asset_application_activity</code>	<code>asset_application_mapping_skill</code>	<code>AssetAppMap</code>
5a	<code>exposure_context_activity</code>	<code>exposure_context_skill</code>	<code>ExposureContext</code>
5b	<code>graph_path_query_activity</code>	<code>graph_path_query_skill</code>	<code>ChainPath[]</code> ($\leq$ <code>max_graph_depth</code>)
5c	<code>data_impact_activity</code>	<code>data_impact_skill</code>	<code>DataImpact</code>
6	<code>risk_scoring_activity</code>	<code>risk_scoring_skill</code> (deterministic scorer)	<code>RiskScore</code> per chain
7	<code>remediation_recommendation_activity</code>	<code>remediation_recommendation_skill</code>	<code>RemediationPlan</code>
8	<code>jira_epic_draft_activity</code> / <code>jira_story_task_draft_activity</code>	<code>jira_*_draft_skill</code>	<code>JiraDraft[]</code> (§7.4)
9	<code>evidence_packaging_activity</code>	<code>evidence_packaging_skill</code>	<code>EvidenceBundle</code> (object store)
10	<code>quality_gate_activity</code>	$\rightarrow$ <code>verification-response-service</code>	<code>QualityGateResult</code>
11	<code>wait_for_approval</code> (signal)	—	<code>ApprovalDecision</code>
12	<code>create_jira_activity</code>	— (deterministic, post-approval only)	Jira issue keys

Steps 5a/5b/5c run in parallel (Temporal `asyncio.gather` over activities). Steps 11–12 execute only if Jira creation was requested and drafts passed the quality gate.

4. Interface Specification

Conventions used throughout:

- **Base path:** `/v1`. JSON request/response, UTF-8.
- **All requests** carry the propagated headers of §6.1 and are authenticated via mTLS + signed execution-context token.
- **Errors** use the uniform error envelope of §6.2.
- **Idempotency:** all POSTs that create state accept `Idempotency-Key`; retried keys return the original result.
- **IDs:** ULIDs unless noted. Timestamps RFC 3339 UTC.

4.1 chat-api-service

POST `/v1/chat/messages/stream` — submit request, stream progress + final answer.

Request:

```
{
  "conversation_id": "01J...",           // optional; new conversation if absent
```

```

"message": "Can CVE-2026-1234 on payments-gw chain to customer data?",
"options": { "request_jira_creation": false }
}

```

Response: `text/event-stream`. Event types:

```

event: workflow.accepted      data: {"execution_id": "...", "workflow": "vulnerability_chaining_workflow@1"}
event: workflow.progress     data: {"step": "graph_path_query", "status": "running", "pct": 55}
event: workflow.approval_required data: {"approval_id": "...", "drafts": [...refs]}
event: message.delta         data: {"text": "..."}
event: message.final         data: {"message_id": "...", "evidence_refs": [...], "confidence": {...}}
event: workflow.failed       data: {"error": {...envelope}}

```

GET /v1/workflows/{execution_id}/status

```

{
  "execution_id": "01J...",
  "workflow": "vulnerability_chaining_workflow",
  "version": 1,
  "status": "RUNNING | AWAITING_APPROVAL | COMPLETED | FAILED | CANCELLED",
  "current_step": "risk_scoring",
  "started_at": "...", "updated_at": "...",
  "approval": { "approval_id": "01J...", "status": "PENDING" }
}

```

4.2 context-api

POST /v1/context/resolve

```

// Request
{ "principal_token": "<idp token>", "conversation_id": "01J...", "channel": "chat" }
// Response
{
  "execution_context": {
    "tenant_id": "t-wf-cyber-01",
    "actor": { "sub": "u:dan", "roles": ["cyber_analyst"], "entitlements_ref": "e:..." },
    "session_id": "01J...", "conversation_id": "01J...",
    "context_token": "<signed JWS, 5m TTL>" // propagated to all downstream calls
  },
  "history": [ { "role": "user", "content": "...", "at": "..." } ],
  "memory_facts": [ { "key": "preferred_env", "value": "prod", "confidence": 0.9 } ]
}

```

GET /v1/conversations?days=30 → { "conversations": [{ "conversation_id", "title", "updated_at" }] }

4.3 orchestrator-service

POST /v1/orchestrate

```

// Request
{ "execution_context": {...}, "message": "...", "options": {...} }
// Response
{
  "intent": "vulnerability_chain_analysis",
  "confidence": 0.94,
  "route": {
    "mode": "governed_workflow", // MVP: only mode
    "workflow_id": "vulnerability_chaining_workflow",
    "workflow_version": 1,
    "inputs": { "entities": { "cve": ["CVE-2026-1234"], "application": ["payments-gw"] } }
  },
  "policy_decision_id": "01J..." // pre-flight decision, §4.5
}

```

Out-of-scope intent → `route.mode = "declined"` with a user-safe explanation; no workflow starts.

4.4 registry-api

Read model for the execution path; write/approval lifecycle is admin-plane (out of MVP runtime scope). All definitions immutable per version.

GET /v1/workflows/{workflow_id}?version=1

```
{
  "workflow_id": "vulnerability_chaining_workflow",
  "version": 1,
  "status": "APPROVED",
  "agent": { "agent_id": "vulnerability_chaining_agent", "version": 1 },
  "steps": [ { "step": 1, "activity": "resolve_entity_activity",
    "skill": "entity_resolution_skill@1",
    "retry_policy": { "max_attempts": 3, "backoff": "exp" },
    "timeout_s": 60 }, ... ],
  "required_tools": ["graph_query_tool@2", "vulnerability_findings_tool@1", ...],
  "output_contract": "vuln_chain_response@1",
  "checksum": "sha256:..."
}
```

GET /v1/agents/{agent_id} → agent definition: allowed skills, model profile ref, prompt refs (versioned), token budget class. **GET /v1/skills/{skill_id}** → skill definition: allowed tools, input/output schema refs, prompt version. **GET /v1/tools/{tool_id}** → tool definition: JSON Schema for args, target adapter, side-effect class (read | draft | write), default guards.

4.5 policy-api

POST /v1/policy/evaluate — the single authorization contract for workflows, tools, and Jira actions.

```
// Request
{
  "decision_request_id": "01J...",
  "execution_context": {...},
  "action": {
    "type": "execute_workflow | invoke_tool | create_jira_epic | create_jira_story | create_jira_task | assign_jira_owner",
    "resource": { "id": "graph_query_tool", "version": 2 },
    "args_digest": "sha256:...",
    "workflow_execution_id": "01J...",
    "approval_id": "01J..." // required for approval-gated actions
  }
}
// Response
{
  "decision_id": "01J...",
  "decision": "ALLOW | DENY",
  "constraints": { // MVP workflow constraints
    "read_only": true,
    "pii_mode": "masked",
    "max_graph_depth": 4,
    "max_tool_calls": 100
  },
  "obligations": [
    { "type": "mask_fields", "fields": ["customer_id", "account_number"] },
    { "type": "audit", "level": "full" },
    { "type": "require_evidence_refs" }
  ],
  "reason_code": "MVP_VULN_CHAIN_POLICY_V1",
  "ttl_s": 300,
  "policy_version": "mvp_vulnerability_chain_policy_v1"
}
```

Rules encoded (from policy `mvp_vulnerability_chain_policy_v1`): - **Allow**:

`execute_vulnerability_chaining_workflow`; `generate_jira_{epic,story,task}draft`. - **Approval-required**:

`create_jira{epic,story,task}`; `assign_jira_owner`; `notify_application_owner`. - **Deny (hard)**: `auto_close_jira`,

`auto_mark_remediated`, `patch_server`, `change_firewall_rule`, `rotate_secret`, `disable_account`. - **require_for_create**:

evidence_refs, risk_score, affected_assets, remediation_reason, owner_verification. - Decisions are cached by consumers at most `ttl_s`; write-class actions are never served from cache.

4.6 workflow-runtime-service

POST /v1/workflows/run

```
// Request
{
  "workflow_id": "vulnerability_chaining_workflow",
  "workflow_version": 1,
  "execution_context": {...},
  "policy_decision_id": "01J...",
  "inputs": {...},
  "idempotency_key": "..."
}
// Response 202
{ "execution_id": "01J...", "temporal_workflow_id": "vc-01J...", "status": "RUNNING" }
```

Validates: registry definition exists & APPROVED; policy decision fresh, ALLOW, and bound to this workflow version. Otherwise 403 with `POLICY_DENIED` / `STALE_DECISION`.

POST /v1/executions/{execution_id}/signal

```
{
  "signal": "jira_approval | cancel | update",
  "payload": {
    "approval_id": "01J...",
    "decision": "APPROVED | REJECTED",
    "approver": "u:jsmith",
    "scope": ["epic:draft-01J...", "story:draft-01J..."], // partial approval supported
    "comment": "Approve epic + patch story; hold data-path story."
  }
}
```

Signal payloads are validated against the pending approval record before delivery to Temporal (defense against out-of-band signal injection).

GET /v1/executions/{execution_id} → Temporal-backed status, step history, iteration count for quality loop.

4.7 agent-runtime-service

POST /v1/agents/execute — invoked as a Temporal activity body.

```
// Request
{
  "execution_context": {...},
  "workflow_execution_id": "01J...",
  "agent_id": "vulnerability_chaining_agent", "agent_version": 1,
  "skill_id": "graph_path_query_skill", "skill_version": 1,
  "inputs": {...},
  "constraints": {...from policy decision},
  "budget": { "max_tokens": 12000, "max_tool_calls": 10 }
}
// Response
{
  "outputs": {...skill output schema},
  "tool_calls": [ { "tool_id": "graph_query_tool", "call_id": "01J...", "status": "OK" } ],
  "usage": { "input_tokens": 3211, "output_tokens": 902, "tool_calls": 3 },
  "evidence_refs": ["ev://bundle/01J.../item/7"]
}
```

Agent-runtime enforces the budget locally and hard-fails on breach (Temporal retry policy decides on resume/abort).

4.8 tool-gateway-service

POST /v1/tools/invoke — sole enterprise egress.

```
// Request
{
  "execution_context": {...},
  "workflow_execution_id": "01J...",
  "call_id": "01J...", // idempotency at gateway level
  "tool_id": "graph_query_tool", "tool_version": 2,
  "args": { "start_nodes": ["asset:srv-01"], "target_labels": ["DataStore"], "max_depth": 4 }
}
// Response
{
  "call_id": "01J...",
  "status": "OK | DENIED | GUARD_BLOCKED | UPSTREAM_ERROR",
  "result": {...},
  "guards_applied": ["graph_depth_guard", "read_only_guard", "pii_mask"],
  "masked_fields": ["customer_id"],
  "policy_decision_id": "01J...",
  "latency_ms": 412
}
```

Gateway pipeline per call: schema validation (registry tool schema) → policy evaluate (or fresh cached read decision) → guard chain (graph guard: depth/label allowlist; SQL guard: read-only, table allowlist, row limits; masking per obligations) → adapter invoke → response filtering → audit emit.

POST /v1/jira/drafts

```
{ "workflow_execution_id": "01J...", "draft_type": "epic | story | task",
  "payload": {...JiraDraft $7.4} }
→ { "draft_id": "01J...", "status": "DRAFTED", "storage_ref": "obj://drafts/01J..." }
```

POST /v1/jira/issues — approval-gated write.

```
// Request
{
  "execution_context": {...},
  "workflow_execution_id": "01J...",
  "approval_id": "01J...",
  "draft_ids": ["01J...epic", "01J...story1"],
  "policy_decision_id": "01J..." // fresh write-class decision required
}
// Response
{ "created": [ { "draft_id": "...", "jira_key": "CYB-4211", "type": "epic" } ],
  "skipped": [ { "draft_id": "...", "reason": "NOT_IN_APPROVAL_SCOPE" } ] }
```

Gateway independently verifies: approval record exists, APPROVED, unexpired, scope covers each draft; policy `require_for_create` fields present in each draft. Any failure → 403, nothing created (all-or-nothing per draft, not per batch).

4.9 cyber-domain-api

Consumed **only via tool-gateway adapters**. Representative endpoints:

ENDPOINT	BACKING TOOL	RETURNS
POST /v1/findings/search	vulnerability_findings_tool	Finding[] — tenant-scoped, severity-filtered
POST /v1/cve/enrich	cve_enrichment_tool	CVSS, EPSS, KEV flag, exploit maturity
POST /v1/assets/search	asset_inventory_tool	Asset[], app/service/env/owner joins
POST /v1/ownership/resolve	application_ownership_tool	Owner team, verified flag
POST /v1/exposure/context	network_exposure_tool	internet/partner/internal/restricted

ENDPOINT	BACKING TOOL	RETURNS
POST /v1/graph/paths	graph_query_tool	ChainPath[] with node/edge evidence
POST /v1/data/classification	data_classification_tool	Data store sensitivity labels
POST /v1/runbooks/search	runbook_retrieval_tool	Runbook chunks + refs
POST /v1/evidence / GET /v1/evidence/{id}	evidence_store_tool	EvidenceBundle refs

POST /v1/graph/paths contract:

```
// Request (post-guard: depth already clamped ≤ 4)
{ "tenant_id": "...", "start_nodes": ["asset:srv-01"],
  "target_labels": ["DataStore", "Identity", "Service"],
  "edge_allowlist": ["RUNS_ON", "CONNECTS_TO", "AUTHENTICATES_AS", "READS", "WRITES"],
  "max_depth": 4, "max_paths": 50 }
// Response
{ "paths": [ { "path_id": "p-01",
  "nodes": [ { "id": "asset:srv-01", "label": "Asset"}, ... ],
  "edges": [ { "type": "CONNECTS_TO", "from": "...", "to": "..."} ],
  "terminal": { "id": "data:cust-db", "label": "DataStore", "classification": "RESTRICTED"},
  "evidence_ref": "ev://graph/p-01" } ],
  "truncated": false }
```

4.10 model-gateway-service

Generic, model-agnostic. Consumed by agent-runtime and verification only.

POST /v1/models/generate

```
// Request
{ "execution_context": {...}, "profile": "reasoning_default | extraction_fast",
  "prompt_ref": "prompt://graph_path_summarizer@3", // registry-versioned; no ad-hoc prompts in prod
  "variables": {...}, "max_tokens": 2048,
  "budget_scope": "workflow_execution:01J..." }
// Response
{ "output": "...", "usage": { "input_tokens": ..., "output_tokens": ... },
  "model_id": "opaque-routing-id", "finish_reason": "stop" }
```

POST /v1/models/embed → { "vectors": [...], "dim": 1024, "usage": {...} } Enforces per-tenant and per-execution token budgets (Redis); 429 with **BUDGET_EXCEEDED** on breach.

4.11 verification-response-service

POST /v1/verify

```
// Request
{
  "execution_context": {...},
  "workflow_execution_id": "01J...",
  "draft_response": {...},
  "artifacts": {
    "evidence_bundle_ref": "obj://evidence/01J...",
    "chain_paths": [...], "risk_scores": [...], "jira_drafts": ["01J...", "01J..."]
  },
  "checks": ["evidence_resolution", "chain_path_validity", "data_impact_grounding",
    "risk_explanation", "jira_draft_contract", "redaction"]
}
// Response
{
  "result": "PASS | FAIL",
  "check_results": [ { "check": "evidence_resolution", "status": "PASS" },
    { "check": "jira_draft_contract", "status": "FAIL",
      "detail": "story draft 01J... missing acceptance_criteria" } ],
  "improvement_actions": [
    { "action": "rerun_skill", "skill_id": "jira_story_draft_skill",
      "inputs_patch": {...}, "reason": "missing acceptance_criteria" }
  ]
}
```

```

    ],
    "redactions_applied": [ {"field": "customer_id", "count": 3} ],
    "final_response": {...} // present only on PASS
  }

```

No tool/database egress (invariant I4). Evidence resolution is performed against artifact refs passed in the request; anything unresolvable fails the check.

4.12 notification-service

POST /v1/approvals

```

// Request
{ "workflow_execution_id": "01J...", "type": "jira_creation",
  "requested_actions": ["create_jira_epic", "create_jira_story", "create_jira_task"],
  "draft_refs": ["01J...epic", "01J...story1", "01J...task1"],
  "approver_group": "cyber_remediation_leads",
  "expires_at": "2026-07-10T00:00:00Z",
  "channels": ["email", "slack"] }
// Response
{ "approval_id": "01J...", "status": "PENDING", "review_url": "https://.../approvals/01J..." }

```

POST /v1/approvals/{approval_id}/decision (called by the approval UI/channel handler)

```

{ "decision": "APPROVED | REJECTED", "approver": "u:jsmith",
  "scope": ["01J...epic", "01J...story1"], "comment": "..."}

```

On decision, notification-service calls workflow-runtime POST /v1/executions/{id}/signal. Approver identity is verified against approver_group membership; self-approval by the requesting actor is rejected (SELF_APPROVAL_DENIED).

GET /v1/approvals/{approval_id} → status, decision, approver, scope, timestamps.

4.13 audit-observability-service

POST /v1/audit/events (also and preferentially via Kafka, §5)

```

{ "event_id": "01J...", "type": "tool.invoked",
  "at": "...", "tenant_id": "...", "actor": "u:dan",
  "workflow_execution_id": "01J...", "decision_id": "01J...",
  "resource": "graph_query_tool@2",
  "outcome": "OK", "detail": { "guards_applied": [...], "latency_ms": 412 },
  "trace_id": "..."}

```

Query surface (GET /v1/audit/events?...) is admin-plane; not exposed to the runtime path.

5. Event Contracts (Kafka)

TOPIC	PRODUCER	KEY	PAYLOAD
audit.events.v1	all services	tenant_id	AuditEvent (§4.13)
tool.calls.v1	tool-gateway	workflow_execution_id	call_id, tool, args_digest, guards, outcome, latency
policy.decisions.v1	policy-api	decision_id	full decision + obligations (immutable record)
workflow.lifecycle.v1	workflow-runtime	execution_id	STARTED / STEP_COMPLETED / AWAITING_APPROVAL / COMPLETED / FAILED / CANCELLED
approvals.v1	notification-service	approval_id	REQUESTED / APPROVED / REJECTED / EXPIRED

TOPIC	PRODUCER	KEY	PAYLOAD
cost.usage.v1	model-gateway	tenant_id	token usage per execution/profile
evidence.events.v1	cyber-domain-api	bundle_id	bundle created/updated (drives OpenSearch indexing)

All events: CloudEvents 1.0 envelope, schema-registry governed, backward-compatible evolution only.

6. Cross-Cutting Contracts

6.1 Propagated execution context

Every inter-service call carries:

HEADER	CONTENT
X-Context-Token	Signed JWS (5 min TTL): tenant_id, sub, roles, entitlements_ref, session_id, workflow_execution_id, token_bind (mTLS cert hash)
traceparent	W3C trace context
X-Request-ID	Per-hop request ULID
Idempotency-Key	State-creating POSTs only

Services validate signature, TTL, and mTLS binding; **no service trusts unsigned tenant/actor fields in the body** — body copies of `execution_context` are informational and must match the token or the request is rejected (`CONTEXT_MISMATCH`).

6.2 Error envelope

```
{
  "error": {
    "code": "POLICY_DENIED",
    "message": "Action create_jira_epic requires an approved approval_id.",
    "request_id": "01J...",
    "decision_id": "01J...",           // when policy-related
    "retryable": false,
    "details": [ { "field": "approval_id", "issue": "missing" } ]
  }
}
```

HTTP	CODES
400	SCHEMA_VALIDATION_FAILED, CONTEXT_MISMATCH
401	CONTEXT_TOKEN_INVALID, CONTEXT_TOKEN_EXPIRED
403	POLICY_DENIED, STALE_DECISION, TENANT_BOUNDARY, SELF_APPROVAL_DENIED, APPROVAL_SCOPE_MISMATCH
409	IDEMPOTENCY_CONFLICT, APPROVAL_ALREADY_DECIDED
422	GUARD_BLOCKED (graph depth, SQL guard, masking failure)
429	BUDGET_EXCEEDED, RATE_LIMITED
502/504	UPSTREAM_ERROR, UPSTREAM_TIMEOUT (retryable per Temporal policy)

6.3 Evidence reference scheme

`ev://bundle/{bundle_id}/item/{n}` and `obj://evidence/{bundle_id}` resolve through `evidence_store_tool` only. Every claim in the final response and every Jira draft field marked evidence-backed must carry at least one resolvable `ev://` ref; verification fails otherwise.

6.4 Jira draft contract (`jira_draft@1`)

Epic:

```
{ "draft_type": "epic",
  "title": "...", "summary": "...",
  "business_impact": "...",
  "risk": { "level": "HIGH", "score": 87.5, "method": "deterministic_v1" },
  "affected_applications": ["payments-gw"],
  "affected_assets": ["asset:srv-01", "asset:srv-02"],
  "remediation_plan": "...",
  "acceptance_criteria": [...],
  "evidence_refs": ["ev://bundle/01J.../item/2", "ev://graph/p-01"],
  "confidence_and_limitations": "..." }
```

Story/task:

```
{ "draft_type": "story",
  "title": "...", "priority": "P1",
  "owner": { "team": "payments-platform", "verified": true } , // or {"unresolved": true}
  "affected": { "assets": [...], "applications": [...] },
  "action_required": "...",
  "acceptance_criteria": [...],
  "evidence_refs": [...] }
```

`policy-api require_for_create` maps directly onto these fields; the gateway rejects creation for any draft missing them.

7. Data Store Ownership

STORE	OWNING SERVICE(S)	CONTENTS
Postgres	context-api, registry-api, policy-api, workflow-runtime, notification	Tenants, sessions, conversations, definitions, policies, execution metadata, approvals, draft metadata, memory facts
Redis	chat-api, tool-gateway, model-gateway	SSE fanout streams, progress cache, rate limits, token budgets, idempotency keys
Temporal	workflow-runtime (exclusive)	Durable workflow state, timers, signals, history
Kafka	audit-observability (consumer of record)	\$5 topics
Graph DB	cyber-domain-api (exclusive)	Asset/app/service/identity/data graph, attack paths
OpenSearch	cyber-domain-api, audit	Runbooks, evidence metadata, audit search
Vector / pgvector	cyber-domain-api	RAG over runbooks, policies, evidence summaries
Warehouse/ Lakehouse	cyber-domain-api	Governed cyber data products
Object store	cyber-domain-api, tool-gateway	Evidence bundles, reports, Jira draft payloads

No service reads another service's store directly. Cross-service data access is API-only.

8. Build Phases → Interface Readiness

PHASE	DELIVERABLE	INTERFACES LIVE
1	Service skeletons + context token + error envelope	§4.1–4.6 stubs, §6.1–6.2
2	Temporal workflow with stubbed tools, synthetic data	§4.6–4.7, workflow.lifecycle events
3	Tool-gateway + cyber-domain integration (graph, findings, assets, classification, evidence)	§4.8–4.9, tool.calls events
4	Verification quality gate + streaming	§4.11, SSE event contract
5	Jira drafts + approval-gated creation	§4.8 Jira endpoints, §4.12, approvals events
6	Audit hardening, tenant boundary tests, load tests	§4.13 full, §5 complete

9. Design Principles (restated as testable assertions)

1. One workflow, one parent agent, approved skills only — orchestrator cannot route to anything not in registry with status APPROVED.
2. Temporal owns durability — no service maintains its own workflow state machine.
3. policy-api answers "allowed now"; registry-api answers "exists and approved" — no service conflates the two.
4. Jira drafts are automatic; Jira creation requires a fresh approval and a fresh write-class policy decision, verified independently at the gateway.
5. verification-response-service proposes; it never executes — zero enterprise egress.
6. Every write-class action is deterministic code, never LLM output applied directly.
7. Fail closed everywhere: unknown capability, stale decision, missing evidence, unverified owner → deny.

Part B — Graph RAG, NL2Query, and the Domain Agent Framework

0. Positioning — nothing bypasses the substrate

All three capabilities are expressed in the existing primitives:

CAPABILITY	EXPRESSED AS	GOVERNED BY
Graph RAG	Skills + read-class tools backed by cyber-domain-api knowledge endpoints	policy constraints (entitlement filter, max_hops, max_chunks) + verification evidence checks
NL2SQL / NL2Cypher	Skill (candidate generation via model-gateway) + tool-gateway guard chain (deterministic validation) + read-class execution tools	SQL guard / graph guard, semantic model allowlist, tenant predicate injection
New domain agents	Registry entries (agent + skills + tools) + policy bundle + Temporal workflow per use case	Same orchestrator → registry → policy → workflow-runtime path as the MVP

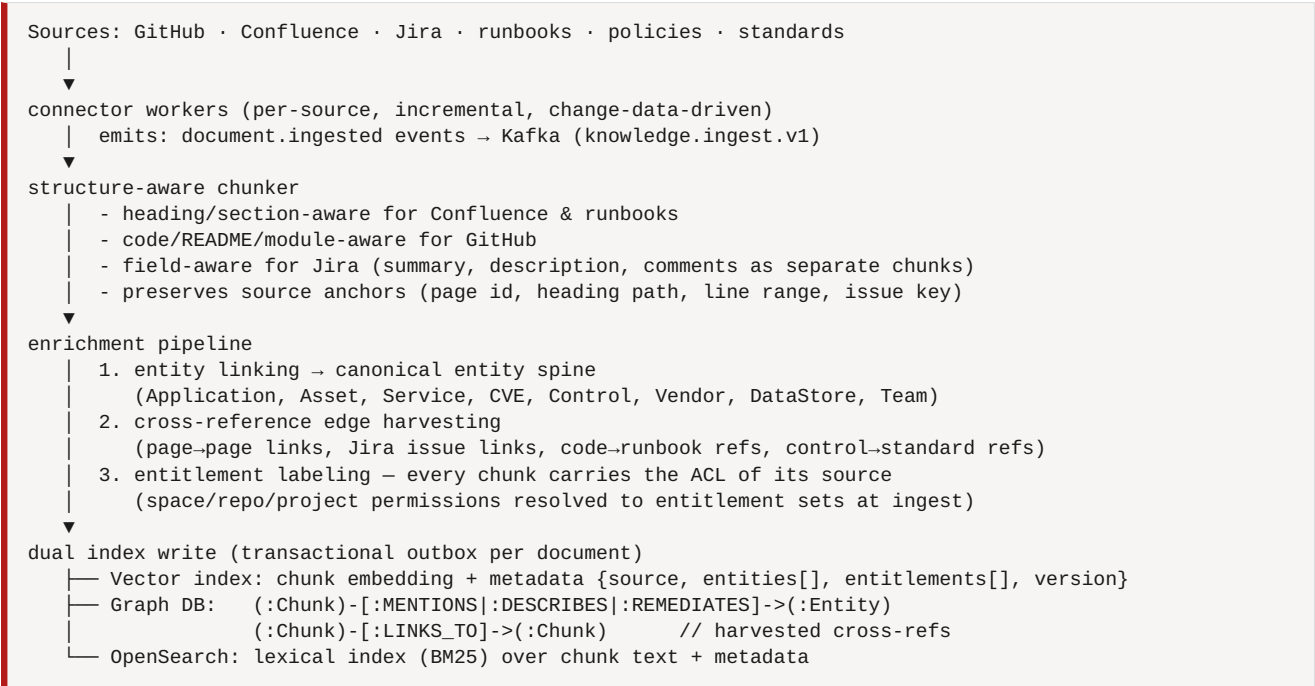
The hard rule carries over from the MVP and is worth stating once, up front:

The LLM never touches a database, a graph, or an enterprise system. It produces text — candidate queries, retrieval plans, summaries. Deterministic code validates, executes, and records. "No inference allowed" in any scoring, authorization, or write path.

1. Graph RAG Subsystem

Two planes: an **ingestion plane** (async, event-driven, offline from the request path) and a **retrieval plane** (skills executing inside workflows, via tool-gateway).

1.1 Ingestion plane



Design points:

- **Canonical entity spine is the join key.** The same `Application`, `Asset`, `CVE`, and `Control` nodes used by vulnerability chaining are the anchor nodes for knowledge chunks. One graph, two edge families (operational topology edges vs. knowledge edges). This is what makes "show me runbooks for the assets on this attack path" a two-hop query instead of a second retrieval system.
- **Entitlements resolved at ingest, enforced at retrieval.** Chunks store entitlement labels; the retrieval tool filters on the caller's `entitlements_ref` (from the execution context token) **pre-ANN** as a metadata filter — never post-hoc filtering of results the caller shouldn't have seen. A chunk with unresolvable ACLs is quarantined, not indexed permissively.
- **Versioning.** Chunks carry source version; superseded chunks are tombstoned, not deleted, so evidence refs in past audit records still resolve.

1.2 Retrieval plane — the Graph RAG skill pattern

`knowledge_retrieval_skill` (and the MVP's `runbook_retrieval_skill`, which becomes a constrained profile of it) executes this sequence, entirely through tool-gateway:

```
question / task context
|
1. entity linking (deterministic first: exact/alias match against spine;
   model-gateway extraction only as fallback, output validated against spine)
|   → seed entities: [app:payments-gw, cve:CVE-2026-1234]
▼
2. hybrid retrieval (single graph_rag_retrieval_tool call)
   a. vector search — entitlement-filtered ANN, top-k per sub-question
   b. lexical search — BM25 for exact identifiers (CVE ids, control ids, hostnames)
   c. graph expansion — from seed entities, bounded hops (≤2) over knowledge
                        edges to pull chunks connected to the entity neighborhood
|   → candidate set with per-channel provenance
▼
3. fusion + rerank (deterministic RRF fusion, then cross-encoder rerank
   via model-gateway; rerank reorders, never adds)
▼
4. evidence packaging — every retained chunk becomes an ev:// item with
   source anchor (page/heading/line/issue key) and retrieval provenance
▼
5. grounded generation via model-gateway (prompt_ref versioned in registry;
   answer must cite ev:// refs per claim)
▼
6. verification-response-service: evidence_resolution + grounding checks
   (a claim without a resolvable ev:// ref fails the gate)
```

Why graph expansion matters in the cyber domain specifically: the question "how do we remediate CVE-2026-1234 on payments-gw" needs the runbook for the *asset class*, the exception policy for the *application*, and the past Jira ticket for the *same CVE elsewhere* — three chunks that share no vocabulary with the question but share graph neighborhood with its entities. Pure vector RAG misses them; the spine finds them.

1.3 Graph RAG interfaces

Tool (registry): `graph_rag_retrieval_tool@1` — side-effect class `read`.

tool-gateway → cyber-domain-api `POST /v1/knowledge/retrieve`

```
// Request (post-guard: hops/chunks already clamped, entitlements injected)
{
  "tenant_id": "t-wf-cyber-01",
  "entitlements_ref": "e:01J...",
  "query": { "text": "remediation steps for CVE-2026-1234 on payments-gw",
    "seed_entities": ["app:payments-gw", "cve:CVE-2026-1234"] },
  "channels": ["vector", "lexical", "graph"],
  "limits": { "max_hops": 2, "max_chunks": 40, "top_k_final": 12 },
  "filters": { "sources": ["confluence", "runbooks", "jira"], "min_version_date": null }
}
// Response
```

```
{
  "chunks": [
    { "chunk_id": "ck-01J...",
      "text": "...",
      "source": { "system": "confluence", "anchor": "SPACE/page-118#patching", "version": "v14" },
      "entities": ["app:payments-gw"],
      "provenance": { "channels": ["graph","vector"], "hop_distance": 1, "rerank_score": 0.91 },
      "evidence_ref": "ev://knowledge/ck-01J..." }
  ],
  "entity_neighborhood": { "nodes": [...], "edges": [...] }, // optional, for path-aware answers
  "truncated": false
}
```

Policy constraints (added to workflow policies that include the skill):

```
graph_rag:
  max_hops: 2
  max_chunks: 40
  entitlement_filter: required # gateway injects; request without it is rejected
  sources_allowlist: [confluence, runbooks, jira, github]
  cross_tenant: deny
```

2. NL2SQL / NL2Cypher / NL2GQL — the governed query pattern

2.1 The pattern

Generation and execution are **separated by the guard chain**. The skill produces a candidate query as inert text; the tool-gateway compiles, rewrites, and validates it deterministically; only the validated, rewritten query reaches an engine.

```
skill (agent-runtime)
|
| 1. GET semantic model from registry-api # the LLM's only view of the schema
|   semantic_model@N: labels/tables, properties/columns, edges/joins,
|   metrics, synonyms, row-level security predicates, few-shot exemplars
| ▼
| 2. model-gateway /v1/models/generate
|   prompt_ref: nl2cypher_generator@N (registry-versioned)
|   variables: {question, semantic_model_slice, prior_guard_feedback?}
|   → candidate query (text) + declared intent {objects, purpose}
| ▼
| 3. tool-gateway /v1/tools/invoke (cypher_query_tool | sql_query_tool | gql_query_tool)
|   args: { "query_text": "...", "declared_objects": [...], "params": {...} }
|
|   GUARD CHAIN (deterministic, AST-based – never regex on query strings):
|   g1. parse to AST (sqlglot for SQL/GQL; openCypher parser for Cypher)
|       parse failure → GUARD_BLOCKED, feedback returned
|   g2. read-only: deny CREATE|MERGE|DELETE|SET|REMOVE|CALL{write} /
|       INSERT|UPDATE|DELETE|DDL|SCRIPT – allow read clauses only
|   g3. object allowlist: every label/table/property/column in the AST must
|       exist in the semantic model version bound to the policy decision
|   g4. tenant scoping: REWRITE the AST to inject the mandatory tenant
|       predicate / RLS predicate – injected, never trusted from the LLM
|   g5. bound enforcement: clamp variable-length paths to max_graph_depth,
|       inject/clamp LIMIT, deny cartesian products, deny unbounded scans
|   g6. cost gate: EXPLAIN / query-plan estimate against per-tool budget;
|       over-budget → GUARD_BLOCKED with plan summary
|   g7. masking: project-level rewrite of PII columns per policy obligations
|   g8. parameterization: literals hoisted to bound parameters
| ▼
| 4. execute via cyber-domain-api (statement timeout, row cap, kill-on-cancel)
| ▼
| 5. evidence: {original_candidate, rewritten_query, plan_hash, row_count,
|   guard_decisions[]} packaged as an ev:// item – the query IS evidence
| ▼
| 6. repair loop: on GUARD_BLOCKED or empty-result-with-suspected-miss,
|   bounded regeneration (max 2) with structured guard feedback in the prompt.
|   Loop is owned by the workflow (Temporal), not by the skill.
```

2.2 Engine routing

Consistent with the federated data architecture:

QUERY CLASS	ENGINE	TOOL	BOUNDS
Tabular / aggregate over cyber data products	BigQuery (warehouse)	sql_query_tool	table allowlist, slot budget, row cap
Bounded graph patterns (≤ 4 hops, known shape)	GQL over BigQuery	gql_query_tool	depth clamp, shape allowlist
Deep traversal, pathfinding, GDS algorithms	Neo4j (incl. ephemeral GDS projections)	cypher_query_tool	depth clamp, GDS algo allowlist, projection TTL

The router is deterministic: the skill declares query class; registry's tool definition binds class $\rightarrow$ engine. The LLM does not choose engines.

2.3 Semantic model (registry-api)

GET /v1/semantic-models/{model_id}?version=N

```
{
  "semantic_model_id": "cyber_topology_sm", "version": 3, "status": "APPROVED",
  "engine_bindings": { "cypher": "neo4j_prod_ro", "sql": "bq_cyber_products", "gql": "bq_cyber_graph" },
  "entities": [
    { "name": "Asset", "table_or_label": "Asset",
      "properties": [ { "name": "hostname", "type": "string", "pii": false },
                     { "name": "owner_email", "type": "string", "pii": true, "mask": "hash" } ],
      "synonyms": [ "server", "host", "machine" ] }
  ],
  "relationships": [
    { "name": "CONNECTS_TO", "from": "Asset", "to": "Asset", "traversal_cost": "high" }
  ],
  "metrics": [ { "name": "critical_finding_count", "expr_ref": "expr://sm/cyber/cfc@1" } ],
  "rls": { "tenant_predicate": "tenant_id = :ctx_tenant" },
  "exemplars_ref": "prompt://nl2cypher_exemplars@5",
  "checksum": "sha256:..."
}
```

The policy decision for a query tool binds (`semantic_model_id`, `version`); guard g3 validates against exactly that version. Schema drift cannot silently widen the LLM's reachable surface.

2.4 Query tool contract (tool-gateway)

POST /v1/tools/invoke with `tool_id`: `cypher_query_tool@1`:

```
// args
{ "query_text": "MATCH (a:Asset {hostname:$h})-[:CONNECTS_TO*1..6]->(d:DataStore) RETURN d",
  "params": { "h": "srv-01" },
  "declared_objects": [ "Asset", "DataStore", "CONNECTS_TO" ],
  "query_class": "deep_traversal",
  "semantic_model": { "id": "cyber_topology_sm", "version": 3 } }

// response
{ "call_id": "01J...", "status": "OK",
  "result": { "columns": [ "d" ], "rows": [...], "row_count": 17, "truncated": false },
  "rewritten_query": "MATCH (a:Asset {hostname:$h, tenant_id:$ctx_tenant})-[:CONNECTS_TO*1..4]->(d:DataStore) RETURN d",
  "guards_applied": [ "ast_parse", "read_only", "object_allowlist", "tenant_inject",
                     "depth_clamp:6-4", "limit_inject:500", "cost_gate:ok", "mask:owner_email" ],
  "plan_hash": "sha256:...",
  "evidence_ref": "ev://query/01J..." }
```

Note the depth clamp in the example: the candidate asked for `*1..6`; the guard rewrote to the policy bound of 4 and recorded the rewrite. Rewrites are visible, auditable facts, not silent behavior.

2.5 Failure semantics

CONDITION	GATEWAY RESPONSE	WORKFLOW BEHAVIOR
Parse failure / disallowed clause / unknown object	422 <code>GUARD_BLOCKED</code> + structured feedback	Repair loop (≤ 2), then fail step with user-safe message
Cost gate breach	422 <code>GUARD_BLOCKED</code> (plan summary)	Regenerate with narrower scope hint
Engine timeout	504 <code>UPSTREAM_TIMEOUT</code>	Temporal retry policy
Empty result	200, <code>row_count: 0</code>	Skill decides: legitimate empty vs. reformulate (counts against repair budget)

3. Domain Agent Framework

3.1 The onboarding contract — an agent is a bundle, not code with credentials

Every new domain agent ships as four registered artifacts. No artifact, no execution:

1. Registry bundle	<code>agent@v</code> , <code>skills@v</code> (I/O schemas, allowed tools, prompt refs), <code>tools@v</code> (arg schemas, side-effect class, adapter binding), semantic model refs, output contract
2. Policy bundle	<code>allow/deny/approval matrix</code> , constraints (depth, budgets, <code>pii_mode</code>), <code>require_for_*</code> field sets, approver groups
3. Workflow definition	Temporal workflow: activity sequence, retry/timeouts, parallelism, approval parks, quality-loop bounds
4. Evaluation set	golden questions + expected evidence shape; gate for registry status <code>APPROVED</code> (and the MRM filing artifact)

The substrate (chat, context, orchestrator, policy, registry, workflow-runtime, agent-runtime, tool-gateway, model-gateway, verification, audit, notification) is **unchanged per agent**. Adding an agent is a data + config event, not a platform change — that is the point of the registry/policy split.

3.2 Agent catalog (post-MVP horizons)

AGENT	WORKFLOW(S)	DISTINCT SKILLS	DISTINCT TOOLS	WRITE ACTIONS (ALL APPROVAL-GATED)
<code>vulnerability_chaining_agent</code> (MVP)	<code>vulnerability_chaining_workflow</code>	per MVP doc	per MVP doc	Jira create
<code>threat_modeling_agent</code>	<code>threat_model_workflow</code>	<code>architecture_ingestion</code> , <code>trust_boundary_mapping</code> , <code>stride_enumeration</code> (over topology graph), <code>control_coverage_gap</code>	<code>design_doc_retrieval</code> (Graph RAG profile), <code>graph_query</code> , <code>control_catalog</code>	threat model doc publish; Jira stories for gaps
<code>control_mapping_agent</code>	<code>control_mapping_workflow</code>	<code>framework_crosswalk</code> (NIST/FFIEC/internal), <code>control_evidence_retrieval</code> , <code>gap_analysis</code>	<code>graph_rag_retrieval</code> , <code>control_catalog</code> , <code>sql_query</code>	crosswalk publish; exception draft
<code>tprm_agent</code>	<code>vendor_risk_workflow</code>	<code>vendor_entity_resolution</code> , <code>third_party_exposure</code> (vendor ↔ app ↔ data edges), <code>questionnaire_evidence</code> , <code>concentration_risk</code>	<code>vendor_registry</code> , <code>graph_query</code> , <code>graph_rag_retrieval</code>	vendor finding create; reassessment ticket

AGENT	WORKFLOW(S)	DISTINCT SKILLS	DISTINCT TOOLS	WRITE ACTIONS (ALL APPROVAL-GATED)
detection_triage_agent	alert_triage_workflow	alert_enrichment, similar_incident_retrieval (vector over incident corpus), blast_radius (graph), triage_recommendation	siem_query (read), graph_query, incident_store	case annotation; escalation ticket — never containment actions in early horizons
risk_ranker_agent	risk_ranking_workflow	portfolio aggregation over other agents' evidence outputs, deterministic ranking	sql_query, evidence_store	ranking report publish

Common structure across all of them: **resolve entities → retrieve (graph + Graph RAG + NL2Query) → deterministic analysis/scoring → recommend → draft → verify → approval-gated write**. The MVP workflow is the template; the catalog is instantiations.

3.3 Scoring stays deterministic — everywhere

`risk_scoring_skill` in the MVP and its analogues in every agent (triage priority, vendor risk tier, gap severity) are **deterministic scorers over LLM-extracted features at most** — and where feasible, over directly queried facts. The SR 11-7 posture:

- LLM may *extract and structure* inputs (with evidence refs per extracted field).
- The score is a versioned, testable function (`method: deterministic_v1`) registered in registry-api with its own eval set.
- Verification checks that every score input resolves to evidence; an unresolvable input voids the score, never defaults it.

3.4 Cross-agent composition — workflows compose, agents don't chat

Multi-agent behavior is expressed as **Temporal parent/child workflows with typed contracts**, never free-form agent-to-agent messaging:

```
cyber_risk_review_workflow (parent)
├─ child: vulnerability_chaining_workflow      → VulnChainReport@1
├─ child: tprm_vendor_exposure_workflow       → VendorExposureReport@1
├─ join: risk_ranking activities (deterministic, over both reports' evidence)
└─ draft → verify → approval → publish
```

Rules: - Every child workflow start passes the **same governance gate** (registry resolve + policy evaluate) as a user-initiated one. Parent context does not confer child authority. - Child outputs are **registered output contracts** (`VulnChainReport@1`), not prose handed between models. The parent consumes structured artifacts + evidence refs. - Budgets compose: parent policy decision carries an aggregate token/tool-call budget; workflow-runtime meters children against it. - The orchestrator is the only component that maps intent → workflow. There is no planner agent selecting agents dynamically in early horizons; the "planner" is the registry of approved compositions. Dynamic planning, if ever introduced, arrives as a new execution mode with its own policy class — not as a relaxation of this one.

3.5 Interface additions (delta to the MVP spec)

INTERFACE	OWNER	PURPOSE
GET /v1/semantic-models/{id}	registry-api	§2.3
GET /v1/output-contracts/{id}	registry-api	Typed cross-workflow artifacts (§3.4)
POST /v1/knowledge/retrieve	cyber-domain-api (via tool-gateway)	§1.3
POST /v1/query/{cypher\ sql\ gql}	cyber-domain-api (via tool-gateway, post-guard only)	§2.4 execution
POST /v1/workflows/run — parent_execution_id param	workflow-runtime	Child workflow lineage + budget inheritance
Kafka knowledge.ingest.v1	connector workers	Ingestion events
Kafka query.audit.v1	tool-gateway	candidate + rewritten query + plan hash per execution
Tools: graph_rag_retrieval_tool, sql_query_tool, gql_query_tool, cypher_query_tool (supersedes graph_query_tool as its guarded generalization), siem_query_tool, control_catalog_tool, vendor_registry_tool	registry-api	Agent catalog enablement

3.6 Phasing (aligned to MVP Phase 6 exit)

PHASE	DELIVERABLE
A (parallel with MVP 3–5)	Knowledge ingestion plane for runbooks + Confluence; graph_rag_retrieval_tool replacing stubbed runbook_retrieval_tool
B	Semantic model v1 + guard chain (g1–g8) + cypher_query_tool/sql_query_tool; vulnerability workflow's fixed graph queries migrate onto the guarded path
C	Second agent (recommend: control_mapping_agent — read-heavy, lowest write risk, exercises Graph RAG hardest) via the onboarding contract, proving "agent = bundle"
D	Cross-agent composition: first parent workflow with typed child contracts; budget inheritance
E	detection_triage_agent + SIEM read integration; tprm_agent

4. Invariants — extended set

The MVP invariants I1–I6 hold unchanged. Added:

#	INVARIANT	ENFORCED BY
I7	LLM-generated queries never execute unrewritten: tenant predicate, depth/limit clamps, and masking are AST rewrites applied by the gateway, recorded in evidence.	tool-gateway guard chain
I8	The LLM's schema visibility equals exactly one approved semantic model version, bound to the policy decision.	registry-api + guard g3
I9	Knowledge retrieval is entitlement-filtered pre-query; no post-hoc filtering of over-fetched results.	cyber-domain-api retrieval engine
I10	Child workflows re-pass the full governance gate; parent authority is not inherited.	workflow-runtime

#	INVARIANT	ENFORCED BY
I11	All scores are versioned deterministic functions; LLM output can be a score <i>input</i> (evidence-backed) but never the score.	registry-api scorer definitions + verification
I12	Cross-agent data exchange occurs only through registered output contracts with evidence refs.	workflow-runtime schema validation

Part C — Agent vs. Workflow: Relationship Model

C.1 The relationship, stated precisely

The workflow abstracts the agent — not the other way around. In the MVP they are deliberately 1:1, but they are distinct abstractions with different jobs, and the cardinality is designed to diverge.

	WORKFLOW	AGENT
Owns	Control flow: activity sequence, retries, timeouts, parallelism, approval parks, quality-loop bounds, budget metering	Reasoning capability: skills, prompt versions, model profile, allowed tools
Executes	The durable skeleton (Temporal)	Inside activity bodies, invoked per-step with bounded inputs and a budget
Contains reasoning	Never	Always
Decides step order	Yes — statically, per registry definition	Never
Routed/ authorized as	The unit the orchestrator routes to and policy-api authorizes	A capability bound to activities within the authorized workflow
Survives	Crashes, restarts, multi-day approval waits	Nothing — stateless per invocation

This is the operational meaning of "governed workflow mode." In a dynamic-agent architecture, the agent is the planner and the workflow is emergent from its decisions. Here it is inverted: Temporal owns the sequence; the agent fills in steps. The agent can fail an activity or return improvement proposals (§ Part A 2.6), but it cannot add steps, reorder them, or reach a write action that is not a declared activity parked behind an approval signal.

C.2 Why two abstractions when the MVP binds them 1:1

Cardinality diverges. A workflow can invoke multiple agents across its activities (a triage workflow using an enrichment agent for steps 1–3 and a recommendation agent for step 7). An agent can be reused across workflows (a knowledge/retrieval agent serving both threat modeling and control mapping). And a workflow can have zero agents — the knowledge ingestion pipeline (Part B §1.1) is a pure deterministic workflow.

RELATIONSHIP	CARDINALITY	EXAMPLE
workflow → agent	0..N per workflow	<code>cyber_risk_review_workflow</code> binds no agent of its own; join logic is deterministic
agent → workflow	1..N reuse	<code>knowledge_retrieval</code> capability shared by threat modeling and control mapping
activity → (skill, agent)	exactly one binding, versioned	registry maps <code>workflow → [activities] → (skill@v, agent@v)</code>

Independent versioning. A prompt bump or model-profile swap is `agent@v+1` without re-filing the process; restructuring the sequence is `workflow@v+1` without touching the reasoning bundle.

Governance granularity. For MRM purposes, the agent (with its prompts and scorers) is the model-adjacent artifact that is filed and eval-gated; the workflow is the process-control artifact that is audited. Collapsing them muddies which change triggers which review.

The approval boundary must not live inside the thing that reasons. Approvals, signals, and write gates are workflow constructs precisely so the LLM-bearing component can never reach them.

C.3 Registry mapping

```
registry-api
  workflow@v
    └─ activities[]
      └─ activity: retrieve_vulnerability_findings
        └─ skill: vulnerability_finding_retrieval_skill@1
          └─ agent: vulnerability_chaining_agent@1
      └─ activity: risk_scoring
        └─ skill: risk_scoring_skill@1          # deterministic scorer
          └─ agent: vulnerability_chaining_agent@1
      └─ activity: create_jira
        └─ approval: jira_approval signal      # no agent – deterministic,
        └─ approval: jira_approval signal      # post-approval write
```

The MVP simply binds every reasoning activity to the one `vulnerability_chaining_agent`. The Part B catalog agents follow the same shape — each headline agent fronts a workflow — but the composition model (Part B §3.4) already breaks strict 1:1: the parent `cyber_risk_review_workflow` has no agent at all, only deterministic join logic over typed child output contracts.